Description des flux de données - pili-pos-android SCE 2.0

Plateforme : Android 13 (API 33), terminal Sunmi D3 Pro (variante avec services Google)

Ce document parcourt une journée de service complète dans un restaurant à travers le système pili-pos-android, en montrant quelles données circulent et où à chaque étape. Il complète DOCUMENT_PROCESSUS_TECHNIQUE_FR.md (qui décrit l'architecture statique) en suivant les données le long d'un parcours utilisateur.

Couche d'interface sur Android. Les règles fiscales sont celles du moteur commun (crates/pos-engine), identique à celui de pili-pos. L'interface est en Kotlin et Compose ; elle appelle le moteur par UniFFI (libpos_ffi.so, docs/android/FFI.md) : chaque commande commands::<module>::do_<fonction> citée ci-dessous (sauf le flux du §7, absent de la build certifiée) est la méthode correspondante de PosEngine (login, addOrderLine, checkout, generateZReport…), table de correspondance de FFI.md §11 ; les événements du moteur (fdm-status, presence-failed…) arrivent par events(). Aucune règle fiscale ni calcul de montant n'est en Kotlin. Là où le texte dit « l'interface », lire cet écran Compose.

1. Début de service - Pointage entrée (Événement S)

Action utilisateur. L'opérateur allume le terminal. L'application démarre seule, en plein écran verrouillé (Lock Task, PLATEFORME_ANDROID_FR.md §3). L'écran de login demande un INSZ. L'opérateur entre 11 chiffres sur le pavé numérique.

Circulation des données.

1. Écran de login (LoginRoute) → commands::auth::do_login(insz, pin) (PosEngine.login).
2. pos-db::queries::users::get_user → ligne SQLite users ; le PIN est comparé à son hash bcrypt.
3. L'état de session de l'application enregistre (insz, name, role).
4. L'interface ouvre l'écran des tables (TablesRoute). Le pointage d'entrée est fait par le moteur, pas par l'UI : do_login (commands/auth.rs) met l'entrée en file et un processus de présence (crates/pos-engine/src/background.rs) appelle commands::social::do_work_in() - un événement fiscal **S** - sauf si l'employé a déjà un poste ouvert (cas d'un écran simplement verrouillé). La connexion n'attend pas le FDM ; si l'entrée n'est pas enregistrée, le moteur émet l'événement presence-failed que l'interface se contente d'afficher :
   - Clé de doublon FDM construite à partir de pos_id, pos_date_time, terminal_id, "WORK_IN", et du prochain compteur séquentiel (lu sous le verrou de signature, voir *STRATEGIE_COMPTEURS_FR.md* §5).
   - Corps JSON canonique POST sur l'endpoint GraphQL du FDM.
   - Sur succès : le FDM renvoie short signature + compteurs ; ligne ajoutée à fiscal_events en une seule transaction ; une ligne staff_shifts ouverte (migration 9) dans la même transaction.
   - Le ticket n'est **pas** imprimé pour les événements S.

2. Ouverture d'une table - Commande (Événement P)

Action utilisateur. L'opérateur tape Table 12 ; la grille du menu se charge. Les clients commandent trois plats ; l'opérateur appuie sur "Envoyer en cuisine".

Circulation des données.

1. L'écran de commande (OrderRoute) appelle commands::order::do_add_order_line une fois par plat. Chaque appel écrit une ligne order_lines avec le nom dénormalisé, le label TVA, le prix unitaire (depuis menu_items), et les éventuels modificateurs.
2. "Envoyer en cuisine" déclenche commands::tables::do_sign_table_order :
   - Construit un OrderInput pour **P** (pro forma) - même forme qu'une vente mais sans paiement.
   - Sérialisé en JSON canonique, stocké dans pending_signs avant l'envoi (voir §3, étape 5).
   - Le FDM signe ; on archive l'événement P.
   - pos-print::bar_ticket::format_bar_ticket (via commands/bar_slip.rs) construit le ticket de commande des seules lignes qui viennent d'être signées, pour les produits marqués

kitchen_printer, et l'envoie à l'imprimante cuisine si elle est configurée (destination TCP distincte de l'imprimante du ticket client principal), sinon à l'imprimante principale.

Le compteur fiscal **P** avance de 1. Le compteur **N** reste intact jusqu'à ce que le client paie effectivement.

3. Le client paie - Vente (Événement N)

Action utilisateur. Le client demande l'addition. L'opérateur tape "Payer" sur la table 12. Choisit "Carte". Le terminal carte renvoie OK. L'opérateur confirme.

Circulation des données.

0. **Choix du support du ticket** (si `pos.json::digital_ticket` est configuré) : l'opérateur sélectionne Papier / E-mail / Les deux avant de taper "Payer" ; pour E-mail, une adresse est saisie et validée à l'écran (`delivery::resolve`). Un choix refusé (adresse invalide, terminal non configuré) s'arrête ici, avant tout appel FDM.
1. Écran d'encaissement (`CheckoutRoute`) → `commands::checkout::do_checkout_split` (`PosEngine.checkout`) avec `{order_id, payment_method: "card", amount_cents, delivery}`.
2. **Pas de pré-contrôle du statut FDM.** La vente passe par `checkout::sell` puis `signing::sign`. Si le FDM n'est pas joignable, `FdmError::outcome()` donne `NotSent` : la ligne `pending_signs` passe à `not_sent`, la commande renvoie « FDM not connected - cannot complete transaction », aucun compteur n'est consommé, aucune ligne métier n'est écrite. En parallèle, le moteur sonde le statut du FDM (`crates/pos-engine/src/commands/fdm_monitor.rs`, §8) ; l'interface n'affiche que l'indicateur et le voile bloquant qu'il lui communique.
3. Construit `SaleInput` à partir des lignes de commande, avec le support résolu à l'étape 0 dans `common.ticketMedium`. Le N ne porte pas de `fdmRef` vers l'événement P : le lien entre les P et le N de la table 12 est la référence du centre de coût, `0{id}` de la commande (`pos-fiscal::event::order_reference`), identique pour tous les événements de la commande (sur place ; une vente à emporter n'a pas de centre de coût).
4. Sous le **verrou de signature** unique (une seule opération signe à la fois), lecture du prochain numéro N (`peek_next`), sérialisation JSON canonique, stockage de la requête exacte dans `pending_signs` (statut `open`), puis appel `FdmClient::sign_sale`. Si une requête précédente est restée sans réponse, elle est d'abord renvoyée à l'identique (§8).
5. Sur succès FDM, dans une seule transaction SQLite :
   - Avance du compteur N (`fiscal_counters`).
   - Ajout de l'événement fiscal à `fiscal_events`.
   - Marquage de la ligne `orders` `status='closed'`.
   - Écriture de la ligne `payments`.
   - Ligne `pending_signs` passée à `signed` (avec `fiscal_event_id`).
   - Si un support numérique a été demandé : ajout d'une ligne `ticket_deliveries` (`delivery::enqueue_in_tx`) - texte du ticket + URL de vérification déjà figés, adresse masquée conservée en clair pour l'envoi à venir.
6. `pos-print::receipt::format_receipt` construit le ticket client :
   - En-tête (nom du restaurant, adresse, TVA).
   - Détail ligne par ligne.
   - Récapitulatif TVA (A=21%, B=12%, C=6%).
   - Bloc de données de contrôle issu de la réponse FDM.
   - QR Code Model 2, ECC M (version 3, 29×29 modules) à partir de l'URL 38 caractères fournie par le FDM.
7. Trame ESC/POS envoyée par TCP au TM-m30III - sauf pour un ticket "E-mail seul" dont l'envoi réussit dans le budget de 8 secondes (`delivery::deliver`), auquel cas aucun papier ne sort. Le tiroir-caisse n'est PAS ouvert (paiement carte).
8. Si un e-mail a été demandé : `delivery::deliver` tente l'envoi tout de suite via `pos_mail::outbox::send_now` (budget 8 s). Sur succès, la ligne `ticket_deliveries` passe à `sent` et l'adresse en clair est aussitôt effacée. Sinon, elle reste `pending` et le papier de secours part (§10 ci-dessous, remise en file en tâche de fond).

4. Rapport X en cours de service

Action utilisateur. Le gérant tape “Rapport X” dans la vue rapports pour voir les totaux en cours.

Circulation des données.

1. Écran des rapports (`ReportsRoute`) → `commands::report::do_generate_x_report`.
2. Les événements de la période de réservation ouverte sont relus dans `fiscal_events` et agrégés par `pos-fiscal::report_totals::turnover`.
3. Construit `ReportTurnoverXInput` (pas de champ booking-date dans les rapports X selon v1.3).
4. Le FDM signe un événement **R** d’opération `ReportTurnoverX` (label R). Archivé dans `fiscal_events`.
5. Ticket imprimé pour le gérant. Les compteurs **ne** sont **pas** remis à zéro.

5. Fin de journée - Rapport Z (Événement R, double enveloppe)

Action utilisateur. Le gérant tape “Clôture journée”.

Circulation des données.

1. Écran des rapports → `commands::report::do_generate_z_report`.
2. La commande émet **deux** événements R de manière atomique :
   - `ReportTurnoverZ` (`ReportTurnoverZInput`, inclut `report_booking_date`).
   - `ReportUserZ` (`ReportUserZInput`).
3. Sous le verrou de signature (une vente en cours est attendue puis comptée dans le Z) :
   - Lecture de deux numéros R consécutifs.
   - Les deux inputs forment **une seule requête** (M184, `ticket_count = 2` dans `pending_signs`), stockée avant l’envoi.
   - Sur succès, dans une seule transaction SQLite : avance des deux compteurs, ajout des deux lignes `fiscal_events`, insertion d’une ligne `z_reports` référençant l’événement turnover, fermeture de la période d’ouverture.
   - Si la requête n’est pas signée (non envoyée ou refusée), rien n’est archivé et les numéros restent libres. Si la réponse est perdue, la requête reste ouverte et est renvoyée à l’identique : le Z est archivé une seule fois, avec ses effets.
   - Le Z automatique (`close_overdue_day`, tick de 60 s) saute son tour si la signature est occupée et revérifie sous le verrou que la journée est toujours en retard : pas de second Z vide.
4. Une fois le Z archivé, sauvegarde de la DB SQLite vers `pos-YYYYMMDD-HHMMSS.db` dans `files/engine/backups/` (heure UTC, répertoire privé de l’application) via l’API Online Backup (`signing::follow_up` → `backup_database_impl`), avant l’impression. Le moteur garde les 7 copies les plus récentes et supprime les plus anciennes (le journal fiscal de `pos.db` n’est jamais supprimé ; commit de cette règle dans le tag de gel : À effectuer (EV-19 : commit de la rétention des sauvegardes)).
5. Deux tickets de rapport Z sont imprimés.
6. Si la section `export` du `pos.json` du lot signé est renseignée, la fin de l’archivage et de l’impression réveille la tâche d’export (§9 ter) ; le Z n’attend ni son résultat ni sa fin. Un Z de formation ne la réveille pas.

6. Fin de service - Pointage sortie (Événement S)

Action utilisateur. L’opérateur touche sa tuile sur l’écran de connexion, puis “Fin de service”.

Circulation des données.

1. Interface → `commands::social::do_end_shift`, qui contrôle le PIN puis appelle `do_work_out`.
2. Événement **S** construit (flag `work_out=true`). Verrou de signature, JSON canonique, FDM, archivage. Le poste ouvert de `staff_shifts` est clos dans la même transaction.
3. Le moteur ferme la session dans tous les cas ; l’interface revient à l’écran de login.

6 bis. Pointage d'un autre employé par le Gérant (Événement S)

Action utilisateur. Le Gérant ouvre "Config", panneau "Employés", et touche "Arrivée" ou "Départ" sur la ligne d'un employé qui n'utilise pas la caisse (profil « Présence seule », cuisine, plonge) ou qui n'a pas pu se pointer lui-même.

Circulation des données.

1. UI → `commands::social::do_clock_in_employee` ou `do_clock_out_employee` (Gérant uniquement). Le moteur refuse, sans rien signer, dans cet ordre : appelant non Gérant, mode formation, utilisateur système, employé inconnu ou désactivé, le Gérant lui-même, arrivée alors que l'employé est en service, départ alors qu'il ne l'est pas.
2. Événement **S** construit au nom de l'**employé** (`employeeId` et `fiscal_events.user_insz` : son INSZ) par `commands::social::sign_work_event`, comme pour la connexion et la fin de service. Verrou de signature, JSON canonique, FDM, archivage.
3. Dans la même transaction, le poste de `staff_shifts` est ouvert ou clos et le Gérant est inscrit dans `started_by` ou `ended_by`.
4. Si le FDM est injoignable ou refuse, la commande répond « échec » avec le texte du FDM : rien n'est archivé, `staff_shifts` ne change pas.
5. L'UI relit les employés et les postes ouverts.

6 ter. Pointage par badge (Événement S)

Action utilisateur. L'employé présente son badge sur le lecteur, depuis n'importe quel écran (option « Lecteur de badges », désactivée par défaut). Un Serveur ou un Apprenti peut aussi s'en servir pour se connecter (§1), et un employé peut terminer son service par badge depuis l'écran de connexion (§6).

Circulation des données.

1. Lecteur → interface. Le lecteur USB (qui se comporte comme un clavier) ou, sur un appareil qui déclare du NFC et si le réglage l'a choisi, le lecteur NFC intégré (`NfcAdapter` en mode lecteur) remet un numéro brut à l'interface. Pour l'USB, l'interface reconnaît une rafale de touches physiques rapides terminée par Entrée (écart et longueur réglables) et la retient : le numéro n'arrive jamais dans un champ de texte. Réglage « Désactivé » : rien n'est lu.
2. Interface → `commands::badges::do_badge_tap` (numéro brut, indicateur « fin de service »), sans session requise : le badge est l'identification. Le moteur normalise le numéro (4 à 64 caractères hexadécimaux, sinon `badge_token_invalid`), le transforme en empreinte HMAC-SHA256 avec la clé de l'appareil (`badge.key`, hors de `pos.db`) ; **le numéro n'est plus jamais utilisé ni écrit** après ce point.
3. Anti-rebond : un second passage de la même empreinte pendant le traitement du premier, ou dans les 10 secondes qui suivent sa fin, est ignoré (aucun journal, aucun événement).
4. Recherche de l'empreinte dans `staff_badges` : inconnue (`badge_unknown`) ou retirée (`badge_revoked`), refus consigné dans `badge_scans`.
5. Décision (même ordre que dans `REPONSES_MANUEL_CERT_FR.md` §III.E) : session ouverte : bascule de présence de l'employé du badge, sans toucher à la session de l'opérateur (son propre badge est refusé, `presence_self`) ; écran de connexion en fin de service : départ seul (`presence_not_in` s'il n'est pas en service) ; écran de connexion avec la connexion par badge : session ouverte comme une connexion par PIN pour un Serveur ou un Apprenti (`logged_in`), PIN demandé pour un Gérant (`pin_required`, aucune session) ; sinon bascule de présence.
6. Bascule de présence : refusée en mode formation (`presence_training`) ; sinon `commands::social::sign_work_eve` construit l'événement **S** au nom de l'**employé du badge** (`signWorkIn` s'il n'est pas en service, `signWorkOut` s'il l'est), exactement celui des flux §1, §6 et §6 bis : verrou de signature, JSON canonique, FDM, archivage dans `fiscal_events` (`user_insz` : l'INSZ de l'employé), ouverture ou clôture du poste de `staff_shifts` dans la même transaction (`started_by` et `ended_by` vides).
7. Le passage est consigné dans `badge_scans` (résultat, INSZ, référence de 12 caractères de l'empreinte, jamais le numéro).
8. Si le FDM est injoignable ou refuse : résultat « échec de présence » avec le texte du FDM, rien n'est archivé, `staff_shifts` ne change pas, et rien n'est mis en tampon. L'interface affiche le texte du FDM.

9. Interface : message (« Arrivée · nom · heure » ou « Départ · nom · heure », rouge pour un refus) et son ; l'activité de l'opérateur n'est pas renouvelée.

7. Exception - Facture B2B (Événement I + Peppol) - hors binaire certifié v1.0.0

Ce flux n'existe pas dans le binaire certifié de la v1.0.0 : le code est sous `#[cfg(feature = "peppol")]` et la build cert ne l'inclut pas (`DOCUMENT_PROCESSUS_TECHNIQUE_FR.md` §8). L'événement I est optionnel dans SCE 2.0 ; il est décrit ici pour une mise à jour ultérieure.

Action utilisateur. Pendant le checkout, l'opérateur active "Générer facture B2B", saisit le n° TVA, le nom, l'adresse et le pays de l'acheteur.

Circulation des données.

1. Le checkout standard se déroule (Événement **N** si vente à emporter, sinon en ligne dans le flux dine-in P/N).
2. Additionnellement, `commands::checkout` invoque `sign_invoice` pour émettre un événement **I** référençant la vente sous-jacente (via `fdmRefs`).
3. Après l'archivage de l'événement fiscal, `commands::invoice::do_issue_b2b_invoice` est appelé avec les détails du fournisseur + de l'acheteur et les lignes de vente :
   - `pos-peppol::ubl::to_ubl_xml` construit un UBL BIS Billing 3.0.
   - `AccessPoint::submit` envoie l'UBL à l'AP configuré (actuellement `LoggingAccessPoint` ; câblage Storecove suivi dans `DECISION_PEPPOL_AP_FR.md`).
   - Ligne ajoutée à `peppol_deliveries` avec statut `accepted` / `failed` et l'UBL complet conservé pour audit.
4. Le ticket imprimé montre le n° TVA de l'acheteur et la phrase "Une facture a été émise et délivrée via Peppol".

8. Exception - FDM injoignable en cours de service

Action utilisateur. Une coupure réseau brève survient entre deux commandes.

Circulation des données.

1. Le sondage de fond du moteur (`crates/pos-engine/src/background.rs` et `commands/fdm_monitor.rs` : toutes les 30 s tant qu'une session est ouverte, recul jusqu'à 5 min tant que le FDM ne répond pas) attrape la déconnexion et classe l'état comme `connected = false`, gravité « bloquante », qu'il communique à l'interface par l'événement `fdm-status` ; le voyant FDM du bandeau passe au rouge et un voile couvre les écrans de vente.
2. Toute tentative de "Payer" part vers `signing::sign` et échoue sans rien écrire (`NotSent`, ligne `pending_signs` à `not_sent`, erreur « FDM not connected »).
3. Il n'existe aucune file d'attente hors-ligne : une commande pro forma (Événement P), une ouverture de tiroir (F) ou toute autre signature est refusée tant que le FDM ne répond pas, et rien n'est archivé.
4. Si la coupure survient **après** l'envoi d'une requête (réponse perdue : délai dépassé, connexion coupée, HTTP 500, corps illisible), la requête reste ouverte dans `pending_signs`. Toute opération de signature la renvoie d'abord à l'identique (même corps, même numéro) ; le FDM, s'il l'avait signée, renvoie sa réponse avec `DUPLICATE_REQUEST` et l'événement est archivé une fois, sous son premier numéro, avec ses effets. Si le renvoi est refusé, la ligne passe à `refused` et son numéro est libre. Sans réponse, l'opération en cours est refusée (`FDM_ANSWER_PENDING`) ; la tâche de fond (60 s) renvoie aussi la requête, la caisse se débloque seule.
5. Une fois le FDM à nouveau opérationnel, le voile se referme automatiquement et l'opérateur retente l'opération. Un nouvel essai est une nouvelle requête (nouveau numéro si la précédente n'a pas été signée) : le POS ne réutilise jamais un numéro avec un contenu différent.

9. Tâche de fond - Envoi différé du ticket numérique

Déclencheur. `run_outbox_once` (`crates/pos-engine/src/commands/delivery.rs`) tourne toutes les 30 secondes (`OUTBOX_INTERVAL`, câblé dans `crates/pos-engine/src/background.rs`), tant qu'un

digital_ticket est configuré.

Circulation des données.

1. Récupération des lignes ticket_deliveries restées bloquées à sending depuis plus de 90 s (plantage applicatif pendant un envoi) → remises à pending.
2. Purge RGPD : toute ligne dont l'adresse en clair a plus de 72 h se fait effacer son recipient (purge_recipients_older_than) ; une ligne failed purgée passe à purged.
3. Sélection des lignes pending dont next_attempt_at est passé (ou jamais fixé), par lots de 20.
4. Pour chacune, réclamation atomique (pending → sending) puis reconstruction de l'e-mail à partir de ticket_text + verification_url déjà stockés (même fonction de rendu qu'à l'encaissement, pos_mail::render::build_email), et appel à l'API Scaleway TEM.
5. Sur succès : ticket_deliveries.status = 'sent', adresse en clair effacée immédiatement.
6. Sur échec retryable (réseau, 5xx, 429) : attempts incrémenté, nouvelle échéance programmée (1 min, 5 min, 15 min, 60 min selon le nombre d'essais), jusqu'à 5 tentatives au total. Au-delà, ou pour une erreur permanente (4xx hors 429), la ligne passe à failed sans être réessayée davantage.

Aucune de ces étapes ne touche à fiscal_events : le ticket fiscal a déjà été signé et archivé à l'encaissement (§3) ; cette tâche de fond ne fait que rattraper la remise numérique promise.

9 ter. Tâche de fond - Export nocturne (facultatif)

Seulement si la section export du pos.json du lot signé est renseignée (EXPORT_NOCTURNE_FR.md).

Déclencheur. Le réveil donné après un Z réel (§5), un passage périodique de crates/pos-engine/src/background.rs et un passage au démarrage. Jamais pour un Z de formation.

Circulation des données.

1. Sur sa propre connexion SQLite **en lecture seule**, sans le verrou de signature ni le verrou des commandes, la tâche lit les Z réels plus récents que le dernier exporté (fichier cursor.json du répertoire d'export).
2. Pour chaque Z : instantané cohérent de pos.db (API Online Backup) dans un répertoire de travail, copie statistique des ventes de la période (opérateurs pseudonymisés, sans INSZ, nom, PIN ni donnée de client), puis chiffrement de l'instantané pour le propriétaire (et le producteur, s'il est configuré). L'instantané en clair est supprimé aussitôt ; l'export complet est déplacé sous exports/<identifiant>/ (files/engine/exports/, jamais sur la carte SD).
3. Suppression des exports locaux plus anciens que keep_days.
4. Envoi HTTPS des trois fichiers dans l'ordre sauvegarde, copie statistique, manifeste, avec le jeton lu dans son fichier et l'empreinte SHA-256 de chaque fichier. Sur échec : nouvel essai après 1, 2, 4 minutes, jusqu'à 60 minutes.
5. Écriture de l'état d'envoi (state.json, status.json), lu par l'écran « Export nocturne » du Gérant, par export_status.

Aucune de ces étapes n'écrit dans pos.db : ni fiscal_events, ni compteur, ni requête au FDM.

9 bis. Commande en ligne (plateforme ou webshop)

Seulement si les commandes en ligne sont activées pour l'installation (COMMANDES_EN_LIGNE_FR.md §2).

Déclencheur. pos_hubrise::poller::run_loop lit les événements HubRise toutes les 30 secondes (POLL_INTERVAL), et au démarrage pos_hubrise::poller::start reprend les commandes nouvelles et relit celles signées depuis deux jours.

Circulation des données.

1. Réception : ligne orders (source = canal, is_fiscal = 0, ouverte par l'identité système) et ligne web-shop_orders (fiscal_state = 'pending'), lignes rapprochées du menu dans lines_json. Statut received mis en file pour HubRise (hubrise_outbox). Rien n'est envoyé au FDM.
2. Acceptation par un employé (do_accept_webshop_order) : réservation pending → signing, écriture des lignes dans le journal order_entries, puis :

- commande payée : `signSale` (N), paiement `APP` ou `ONLINE`, `costCenter` `PLATFORM` ou `WEBSHOP`, remises en `priceChanges` ; ticket de caisse TVA imprimé ;
- commande à payer : `signOrder` (P) avec le même `costCenter`.

3. Dans la transaction qui archive l'événement (`fiscal_events`) : `webshop_orders.fiscal_state = signed_n` ou `signed_p`, `orders.is_fiscal = 1`, statut `accepted` mis en file pour HubRise.
4. Commande à payer, à la remise : `signSale` (N) avec le même `costCenter`, paiement encaissé à la caisse ; statut `completed`.
5. `pos_hubrise::outbox::run_loop` envoie les statuts en file, dans l'ordre par commande, avec nouvelles tentatives.

Si le FDM échoue à l'étape 2, la commande revient à `pending` et rien n'est mis en file.

10. Récapitulatif de persistance

Action	Tables touchées
Login	`users` (lecture)
Pointage entrée/sortie	`staff_shifts` (ouverture / clôture), `fiscal_events` (append S)
Pointage par badge	`staff_shifts` (ouverture / clôture), `fiscal_events` (append S), `badge_scans` (append) ; `staff_badges` (lecture) ; fichier `badge.key` (lecture)
Lier / retirer un badge, régler le lecteur	`staff_badges` (insertion, ou une seule date de retrait), `badge_log` (append), `badge_settings` (mise à jour)
Ajout ligne commande	`orders` (insert/update), `order_lines` (insert)
Envoi cuisine	`fiscal_events` (append P), `order_entries` (lignes marquées envoyées)
Paiement	`fiscal_events` (append N), `payments` (insert), `orders` (close), `fiscal_counters` (advance N)
Rapport X	`fiscal_events` (append R, `ReportTurnoverX`)
Rapport Z	`fiscal_events` (append R, `ReportTurnoverZ` + `ReportUserZ`), `z_reports` (insert), fichier de backup DB (7 conservés)
Facture B2B (hors binaire certifié v1.0.0)	`fiscal_events` (append I), `peppol_deliveries` (insert) ; les tables existent mais ne sont jamais écrites par la build cert
Remboursement	`fiscal_events` (append N M111/M112 référençant l'original)
Ouverture tiroir (F-WAR)	`fiscal_events` (append F)
Mouvement caisse	`fiscal_events` (append F) ; aucune ligne `payments` n'est écrite
Ticket signé avec support numérique	`ticket_deliveries` (append, même tx que `fiscal_events`)
Impression d'un original (vente, remboursement, ou rattrapage)	`original_prints` (une ligne, jamais deux, par événement)
Envoi/réessai/purge du ticket numérique (tâche de fond)	`ticket_deliveries` (écriture des colonnes de statut, jamais `fiscal_events`)
Export nocturne (tâche de fond, facultative)	Aucune table : lecture seule de `pos.db` ; écriture de fichiers sous `exports/` uniquement
Import du lot de configuration signé (§11)	Aucune table, sauf le chargement de `seed.sql` dans une base qui n'existe pas encore ; fichiers sous `provisioning/`
Export du Gérant pour le SPF (§12)	Aucune table : une copie cohérente de `pos.db` ; fichiers écrits sur le support amovible

Toute signature s'exécute sous un seul verrou. Une transaction SQLite unique, ouverte après la réponse du FDM, englobe l'avance du compteur, l'archivage de l'aller-retour FDM, la clôture de la ligne `pending_signs` et tout effet de bord sur la commande. Soit tout commit, soit tout rollback ; si le rollback suit une signature réussie, la ligne `pending_signs` reste ouverte et l'opération suivante archive l'événement.

11. Installation - Import du lot de configuration signé

Action. À l'installation, ou pour changer la configuration, l'installateur choisit dans le sélecteur de fichiers le dossier du lot (clé USB ou carte SD). Le lot contient `pos.json`, les fichiers qu'il nomme (jeton du FDM, matériel mTLS, clés publiques et jeton de l'export), éventuellement `seed.sql`, `MANIFEST.json` et sa signature `MANIFEST.json.minisig`.

Circulation des données (`docs/android/APP.md` §3) :

1. Le moteur est fermé, puis le dossier est copié dans `provisioning/staging-<millis>` du stockage privé, avec des limites (fichiers ordinaires, noms simples, profondeur, nombre et taille bornés, aucun lien symbolique).
2. `verifyBundle` vérifie la signature minisign du manifeste avec la clé publique intégrée à l'APK, puis la taille et le SHA-256 de chaque fichier ; un fichier non listé est refusé. Sans clé publique, tout lot est refusé.
3. `checkConfigBundle` relit `pos.json` (section `export` comprise).
4. `seed.sql` n'est chargé (`installInitDb`) que dans une base qui n'existe pas encore ; le moteur le refuse dès que la base contient un événement fiscal.
5. Le dossier devient `provisioning/bundle/` et `record.json` du dossier `provisioning/` garde le deviceId, le SHA-256 du manifeste et l'heure ; le moteur est rouvert. À chaque démarrage, avant d'ouvrir le moteur, l'application compare le numéro de série lu par le propriétaire de l'appareil à celui de `record.json` : s'ils diffèrent, le moteur n'est pas ouvert (`docs/android/APP.md` §3, `PLATEFORME_ANDROID_FR.md` §4).

Un échec à une étape garde le lot précédent et ne laisse aucun dossier temporaire. Cette étape n'écrit aucun événement fiscal et n'appelle pas le FDM.

12. Contrôle - Export des données pour le SPF Finances (Art. 16 et 102)

Action utilisateur. Un Gérant ouvre « Export pour le SPF » des réglages, branche une clé USB ou une carte SD, choisit un dossier et lance l'export (`SECURITE_BD_FR.md` §8).

Circulation des données (`docs/android/PLATFORM.md` §7) :

1. Contrôles à chaque lancement : le support est monté, le moteur prêt, l'utilisateur est Gérant (`getCurrentUser`), aucune commande ni vente à emporter ouverte (`listOpenOrders`, `listOpenTakeaways`). Sinon rien n'est écrit.
2. Le moteur produit une copie cohérente de `pos.db` (`backupDatabase`, API Online Backup) ; elle reste dans `files/engine/backups/`.
3. Sont copiés dans `pili-export-<date>-<heure>-<deviceId>` : cette copie, les 7 sauvegardes les plus récentes, les journaux, le journal d'audit de la plateforme, `identity.json` (empreintes de l'application), `LISEZMOI.txt`. Les copies de l'export nocturne (`exports/`) n'en font pas partie.
4. Chaque fichier est lu par blocs de 1 Mio, haché en SHA-256, écrit, synchronisé, relu et haché à nouveau ; une différence fait échouer l'export. `MANIFEST.sha256` est écrit en dernier.

L'export n'écrit aucune table de `pos.db`, ne signe rien et ne supprime rien, ni sur le support ni dans le terminal.